

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità nel caso pessimo del seguente algoritmo MYSTERY:

Algorithm 1: MYSTERY(INT n) $\rightarrow$ INT

```

L = NIL
INSERT(L, n)
while L  $\neq$  NIL do
    x = HEAD_DELETE(L)
    if x  $\leq$  1 then
        return x
    else
        INSERT(L, x - 1)
        INSERT(L, x/2)           // Integer division x/2

```

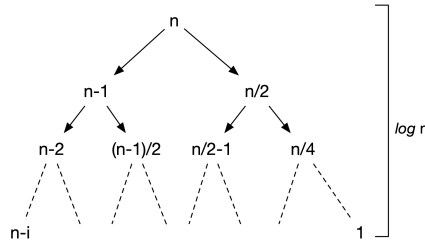
dove L è una lista concatenata semplice dotata di puntatori sia alla testa che alla coda, in modo che entrambe le operazioni INSERT e HEAD_DELETE abbiano un costo costante. In particolare, calcolare la complessità di MYSTERY nei due casi:

- a) INSERT inserisce un valore in testa (L si comporta come una pila)
- b) INSERT inserisce un valore in coda (L si comporta come una coda)

Suggerimento: *disegnare* con l'ausilio di un albero binario gli interi generati dall'algoritmo a partire da n e tracciare su tale albero la sequenza di inserimenti/estrazioni.

Soluzione.

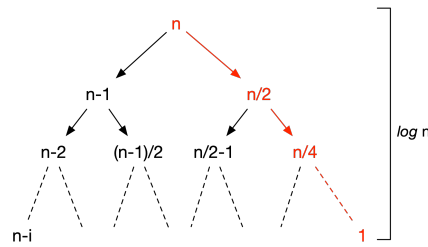
La complessità di MYSTERY dipende dal numero di iterazioni del ciclo *while*, che a sua volta dipende dal numero di estrazioni necessarie per ottenere un valore $x \leq 1$ dalla struttura dati L (si noti il comando *return* all'interno del ciclo *while*). Ad ogni iterazione, per ogni valore x estratto, l'algoritmo inserisce in L i due nuovi valori $x - 1$ e $x/2$. La serie di valori generati può essere rappresentata con un albero:



Il numero di iterazioni del ciclo *while* dipende quindi dall'ordine in cui questi valori vengono estratti da *L*.

- a) Se *L* si comporta come una pila (inserimento e rimozione in testa), l'ultimo elemento inserito è il primo ad essere estratto. Ad ogni iterazione vengono inseriti prima $x - 1$ e poi $x/2$. Poiché l'ultimo elemento inserito è il primo ad essere estratto, il successivo valore estratto sarà $x/2$. In questo caso, l'algoritmo genera la seguente sequenza di estrazioni:

$n, n/2, n/4, \dots, 1$



Un valore ≤ 1 viene quindi raggiunto dopo un numero logaritmico di iterazioni:

valore	n	$n/2$	$n/4$	$\dots$	$n/2^{i-1}$
iterazione	1	2	3	$\dots$	i

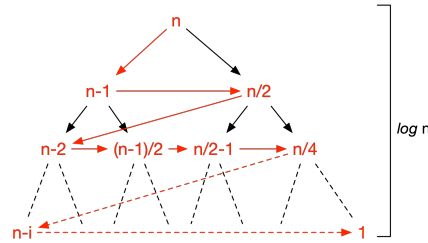
Il ciclo termina quando:

$$n/2^{i-1} \leq 1 \Rightarrow n \leq 2^{i-1} \Rightarrow \log_2 n \leq \log_2 2^{i-1} \Rightarrow i \geq \log_2 n + 1 \Rightarrow i = \Theta(\log n)$$

Pertanto, nel caso in cui *L* sia gestito come una pila, il costo di MYSTERY è $\Theta(\log n)$.

- b) Se *L* si comporta come una coda (inserimento in coda e rimozione in testa), il primo elemento inserito è il primo ad essere estratto. In questo caso, la sequenza di inserimenti (e quindi di estrazioni) è:

$n, n-1, n/2, n-2, (n-1)/2, n/2-1, n/4, \dots, n-i, \dots, 1$



Anche in questo caso il primo valore 1 (che si trova a profondità $\log_2 n$) viene raggiunto seguendo il ramo ottenuto applicando ripetutamente la trasformazione $x \rightarrow x/2$. Tuttavia, prima di raggiungere tale livello, la coda deve estrarre tutti i valori generati nei livelli precedenti dell'albero. Il numero totale di nodi presenti fino alla profondità $\log_2 n$ è:

$$1 + 2 + 4 + \dots + 2^{\log_2 n} = \sum_{i=0}^{\log_2 n} 2^i = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = 2n - 1 = \Theta(n)$$

Pertanto, nel caso in cui *L* sia gestito come una coda, il costo di MYSTERY è $\Theta(n)$.

2. Si consideri una Tabella Hash *T* di dimensione $m = 11$, inizialmente vuota, con funzione hash

$$h'(k) = k \bmod m$$

e la seguente sequenza di operazioni:

- 1) insert 39 2) insert 15 3) insert 18
- 4) insert 17 5) insert 28 6) delete 17
- 7) insert 6

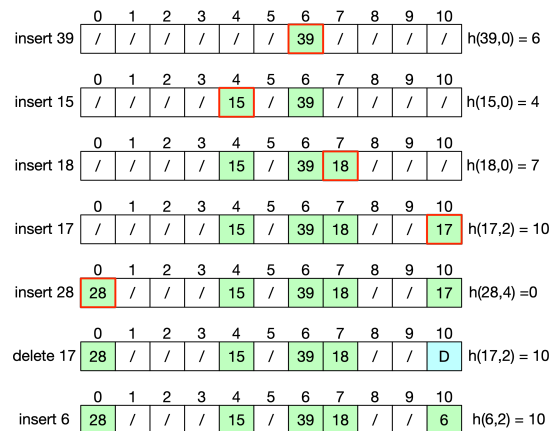
a) Mostrare lo stato della tabella hash T dopo l'esecuzione delle precedenti operazioni, assumendo che le collisioni siano risolte mediante *indirizzamento aperto* con *probing quadratico*:

$$h(k, i) = (h'(k) + i^2) \bmod m$$

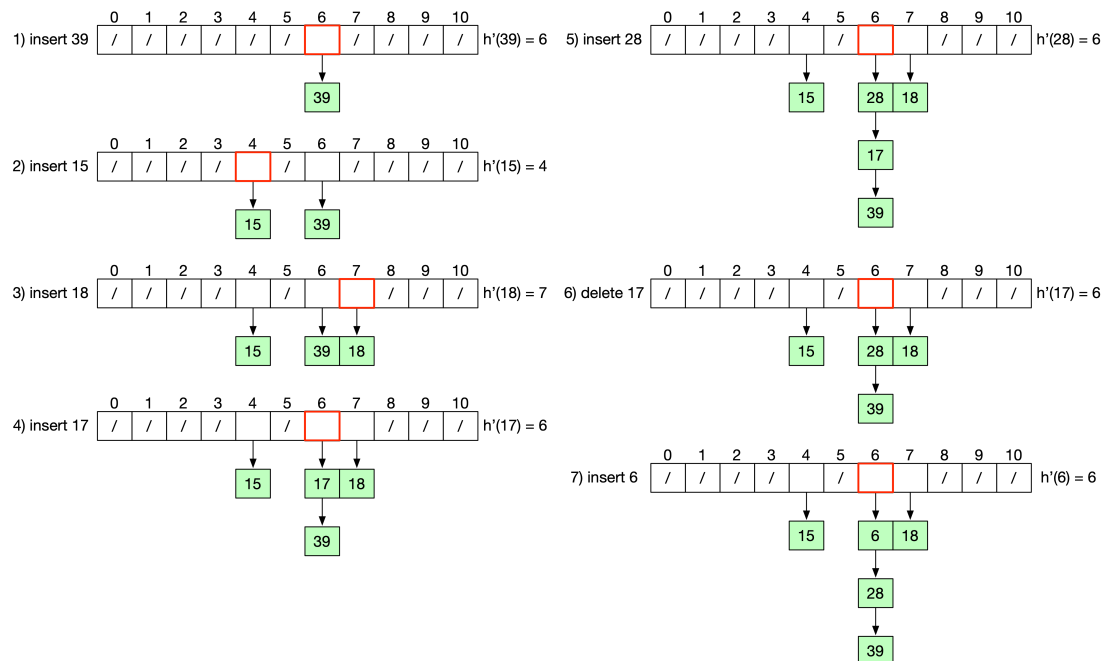
b) Mostrare lo stato della tabella hash T dopo l'esecuzione delle precedenti operazioni, assumendo che le collisioni siano risolte mediante *concatenamento*.

Soluzione

a) *Indirizzamento aperto*



b) *Concatenamento*



3. Progettare un algoritmo che dato un array di numeri $A[1..n]$ ed un numero naturale m stampa gli m numeri più grandi. Si richiede che tale algoritmo sia efficiente nel caso in cui il numero m sia piccolo, in particolare, sotto l'assunzione $m < \frac{n}{\log n}$ l'algoritmo dovrebbe avere costo computazionale in tempo $T(n, m) = O(n)$.

Soluzione Un modo possibile per cercare gli m numeri più grandi di un array di lunghezza n è trasformare l'array in max-heap e poi eseguire m estrazioni dall'heap. Considerando che il

costo della trasformazione in heap (tramite l'algoritmo studiato a lezione) è $O(n)$ e ogni singola estrazione (tramite l'algoritmo studiato a lezione) ha costo $O(\log n)$, il costo complessivo di questo approccio sarà $O(n) + m \times O(\log n)$. Se sappiamo che $m < \frac{n}{\log n}$, allora tale costo sarà $O(n) + O(\frac{n}{\log n})O(\log n) = 2 \times O(n) = O(n)$. Tale idea viene implementata dall'Algoritmo 2 dove le procedure **HEAPIFY**, **FINDMAX** e **DELETMAX** sono quelle studiate a lezione.

Algorithm 2: **EMMEMASSIMI**(**NUMBER** $A[1..n]$, **NATURAL** m)

```

// Si trasforma A in un max-heap
HEAPIFY(A)
// Si stampano gli m massimi
for i = 1..m do
    PRINT(FINDMAX(A))
    DELETMAX(A)

```

4. Un campo da gioco di un gioco da tavolo corrisponde a un grafo non orientato. Ogni giocatore ha una posizione di inizio e di fine gioco, identificate rispettivamente da un vertice di partenza ed un vertice di arrivo. Ad ogni mossa, un giocatore può spostarsi dal vertice attuale ad un vertice adiacente. Esistono alcuni vertici chiamati vertici intermedi: lo scopo del gioco è partire dal vertice di partenza e arrivare al vertice di arrivo attraversando almeno un vertice intermedio. Bisogna calcolare il numero minimo di mosse necessarie.

Progettate quindi un algoritmo che dato in input il grafo non orientato $G = (V, E)$ del campo da gioco, il vertice di partenza $s \in V$, il vertice di arrivo $t \in V$, e i vertici intermedi $I \subseteq V$, restituisce la lunghezza del cammino minimo che inizia in s , termina in t , e attraversa almeno un vertice in I .

Soluzione Il cammino di lunghezza minima che parte da s , arriva in t e attraversa almeno un vertice in I , passa per il vertice in I che ha la minima somma della sua distanza da s e della distanza di t da esso. Si noti che si tratta di un grafo non orientato, quindi la distanza di un vertice v_1 da v_2 e la distanza di v_2 da v_1 corrispondono. Dobbiamo quindi trovare il vertice in I che ha la minima somma della sua distanza da s e della sua distanza da t . Le distanze dei vertici da s e da t si possono calcolare eseguendo due visite *BFS* partendo da s e t , rispettivamente.

Questo approccio viene utilizzato dall'Algoritmo 3: si richiama due volte una funzione ausiliaria che esegue la *BFS*, la prima volta partendo da s , la seconda volta partendo da t , e restituisce il vettore delle distanze. Come solitamente fatto a lezione, assumiamo che i vertici del grafo siano numeri interi che possiamo utilizzare come indici all'interno di un vettore. Successivamente si controllano tutti i possibili vertici intermedi calcolando la somma minima delle relative distanze. Ognuna delle due *BFS* ha un costo computazionale $O(n + m)$, con $n = |V|$ e $m = |E|$, assumendo implementazione del grafo tramite liste di adiacenza. La ricerca della somma minima ha un costo $O(|I|)$, ma visto che $I \subseteq V$ abbiamo $|I| \leq |V|$ quindi tale ricerca ha costo $O(n)$, visto che $n = |V|$. Complessivamente il costo risulta essere $2 \times O(n + m) + O(n) = O(n + m)$.

Algorithm 3: MINIMEMOSSE(GRAPH $G = (V, E)$, VERTEX s , VERTEX t , SET[VERTEX] I) $\rightarrow$ NATURAL

NATURAL $D_s[1..G.vertexNum()] \leftarrow \text{BFS}(G, s)$

NATURAL $D_t[1..G.vertexNum()] \leftarrow \text{BFS}(G, t)$

NATURAL $min \leftarrow \infty$

for $i \in I$ **do**

if $D_s[i] + D_t[i] < min$ **then**
 $min \leftarrow D_s[i] + D_t[i]$

return min

$\text{BFS}(\text{GRAPH } G = (V, E), \text{VERTEX } s) \rightarrow \text{NATURAL}[]$

NATURAL $D[1..G.vertexNum()]$

QUEUE $q \leftarrow \text{new QUEUE}()$

for $x \in V$ **do**

$D[x] \leftarrow \infty;$

$D[s] \leftarrow 0$

$q.enqueue(s)$

while $\text{not } q.isEmpty()$ **do**

$u \leftarrow q.dequeue()$
 for $w \in u.adjacents()$ **do**
 if $D[w] == \infty$ **then**
 $D[w] \leftarrow D[u] + 1$
 $q.enqueue(w)$

return D
